

Proyecto Final Integrador: Arquitectura, Seguridad y Automatización de una API de Predicción en la Nube

1. Descripción General del Proyecto

En la actualidad, las aplicaciones no operan de forma aislada; viven en ecosistemas dinámicos en la nube que requieren ser rápidos, seguros y escalables. Durante este seminario, los estudiantes actuarán como **Arquitectos Cloud** para una empresa simulada de tecnología.

El Caso de Estudio: "FinTech Nova" y la API de Riesgo Crediticio

El Contexto (El Roleplay para los estudiantes): La firma consultora Roslaysoft ha cerrado un contrato con **FinTech Nova**, una *startup* financiera de rápido crecimiento que ofrece microcréditos 100% digitales. Actualmente, FinTech Nova tiene su motor de "Evaluación de Riesgo Crediticio" (Credit Scoring) corriendo en un servidor local antiguo que se satura los fines de semana.

Han contratado a los estudiantes para que tomen el código base de ese motor (la API en FastAPI) y diseñen una arquitectura en la nube que sea segura, escalable y automatizada.

La Lógica de Negocio (¿Qué hace la API?)

La API que les entregarás simulará recibir datos de un solicitante de crédito y devolverá una evaluación predictiva de su nivel de riesgo.

- **Endpoint Principal (/evaluar-riesgo):** Recibe un *payload* (un archivo JSON) con datos básicos como edad, ingresos mensuales y deudas actuales. La API procesa esto con una lógica matemática muy simple (simulando un modelo predictivo) y devuelve si el crédito es "Aprobado", "Rechazado" o "En Revisión".
- **Endpoint Administrativo (/status):** Devuelve la salud del sistema (para que los estudiantes hagan el script de monitoreo en la Unidad 3).
- **Endpoint Vulnerable (/datos-financieros/{id_cliente}):** Un endpoint legado que, si se le pasa el ID de un cliente, devuelve todo su historial de transacciones. **Aquí está la trampa:** inicialmente, este endpoint no tiene autenticación.

Cómo encaja la Lógica del Negocio en los Laboratorios

Al darle esta narrativa financiera, cada laboratorio cobra sentido empresarial:

- ❖ **Laboratorio 1 (Arquitectura Base):** FinTech Nova necesita ver su motor operando en la nube lo antes posible. Los estudiantes levantan la API en Codespaces y generan el diagrama que muestra cómo los usuarios de la app móvil de la FinTech se conectarán a este servicio.
- ❖ **Laboratorio 2 (El Incidente de Seguridad):** Un escáner automatizado de Roslaysoft detecta que el endpoint `/datos-financieros` está expuesto a internet. Si la Superintendencia Financiera lo descubre, FinTech Nova recibirá una multa millonaria. Los estudiantes deben aplicar el control de acceso (Token/Autenticación) de inmediato y entregar el reporte de auditoría.
- ❖ **Laboratorio 3 (Eficiencia Operativa):** Los desarrolladores de FinTech Nova se quejan de que "en sus máquinas funciona, pero en el servidor no". La solución de la consultoría es empaquetar la API en un contenedor Docker. Además, como son un banco, exigen que un script automatizado registre la salud del sistema cada hora en un archivo `log` para auditorías.
- ❖ **Laboratorio 4 (Pase a Producción):** Para asegurar que futuras actualizaciones del motor de crédito no causen caídas del servicio (como le pasaba a la FinTech al inicio), los arquitectos configuran el pipeline de integración continua (GitHub Actions) y entregan la arquitectura final.

Este enfoque convierte un simple ejercicio de programación y redes en un reto de arquitectura de soluciones reales, integrando las necesidades de negocio (Finanzas/IA) con las exigencias operativas (Disponibilidad y Seguridad).

El Reto:

Se entregará a cada grupo de trabajo el código fuente base de una **API de Análisis y Predicción de Datos** construida en Python (FastAPI). El objetivo del proyecto no es programar la aplicación desde cero, sino diseñar su arquitectura, desplegarla en un entorno en la nube (GitHub Codespaces), blindarla contra vulnerabilidades de seguridad y automatizar su ciclo de vida y monitoreo mediante prácticas modernas de infraestructura y CI/CD.

El proyecto se construirá de forma estrictamente **iterativa**, donde cada laboratorio entregable agregará una nueva "capa" de madurez a la arquitectura.

2. Integración Iterativa de los Laboratorios (Fases del Proyecto)

El proyecto se divide en cuatro fases consecutivas que se evaluarán al final de cada unidad del seminario:

Fase 1: Despliegue Base y Diagramación Arquitectónica (Laboratorio 1)

- **Contexto:** La empresa requiere que la nueva API de predicción sea desplegada rápidamente en un entorno de desarrollo en la nube para validar su funcionamiento básico.
- **Acción del Estudiante:** * Realizar un *fork* del repositorio base proporcionado por la docente.

<https://github.com/RoslayBautista/SeminarioSanmateo>

- Inicializar el entorno de desarrollo efímero utilizando GitHub Codespaces.
- Ejecutar la API manualmente (`uvicorn main:app --host 0.0.0.0 --port 8000`) y exponer el puerto públicamente para interactuar con la interfaz Swagger UI autogenerada.
- Diseñar el diagrama de la arquitectura actual ("As-Is").
- **Entregable al cierre de la Unidad 1:**
 - Diagrama arquitectónico formal (exportado en imagen).
 - URL pública del Codespace demostrando la API operativa.

Fase 2: Auditoría de Ciberseguridad y Hardening (Laboratorio 2)

- **Contexto:** El departamento de seguridad ha detectado que la API expone información confidencial a través del endpoint `/datos-sensibles`. Es urgente auditar el sistema y aplicar controles de acceso.
- **Acción del Estudiante:** * Instalar y ejecutar herramientas de escaneo de red (ej. Nmap) desde la terminal del Codespace para documentar los puertos y servicios expuestos.
 - Modificar el código base de `main.py` para implementar un control de seguridad en FastAPI (ej. requerir autenticación básica o un token de validación) protegiendo el endpoint vulnerable.
- **Entregable al cierre de la Unidad 2:**
 - Documento de auditoría (archivo Markdown) con la evidencia del escaneo inicial y la vulnerabilidad encontrada.

- Demostración de la API actualizada, la cual debe rechazar peticiones no autorizadas devolviendo un error `401 Unauthorized`.

Fase 3: Contenerización y Scripting Administrativo (Laboratorio 3)

- **Contexto:** Para garantizar que la API funcione igual en cualquier entorno y evitar tareas manuales repetitivas, la dirección de TI exige que la aplicación sea empaquetada y que su estado de salud sea monitoreado automáticamente.
- **Acción del Estudiante:** * Crear un `Dockerfile` optimizado (usando `python:3.9` como imagen base) para empaquetar la aplicación y sus dependencias.
 - Desarrollar un script en Python (ej. `monitor_health.py`) que consulte automáticamente el endpoint `/status` de la API, verifique su estado y genere un archivo de `log` local con la fecha y hora de la comprobación.
- **Entregable al cierre de la Unidad 3:**
 - El repositorio actualizado con el `Dockerfile` y el script de monitoreo.
 - Demostración de la API ejecutándose dentro del contenedor Docker (`docker run`) en lugar de la ejecución manual.

Fase 4: Despliegue Continuo y Sustentación (Laboratorio 4)

- **Contexto:** El proyecto está listo para pasar a producción. Se debe garantizar que cualquier cambio futuro en el código no rompa el sistema antes de ser desplegado.
- **Acción del Estudiante:** * Implementar un *pipeline* básico de CI/CD utilizando **GitHub Actions**.
 - Configurar un archivo `.yaml` que automatice la validación y construcción de la imagen de Docker cada vez que se realice un *push* al repositorio.
- **Entregable al cierre de la Unidad 4 (Sustentación Final):**
 - - El repositorio debe mostrar el *workflow* de GitHub Actions ejecutándose exitosamente (indicador en verde).
 - **Sustentación ante el panel:** El grupo presentará su diagrama de arquitectura final (actualizado con la contenedorización), demostrará la seguridad del endpoint y explicará el código de sus rutinas de automatización

3. Condiciones de Entrega y Herramientas

- **Modalidad:** Grupal (2 a 4 estudiantes).
- **Infraestructura Requerida:** 100% Cloud (GitHub Codespaces). **No se requiere instalar ningún software de virtualización o lenguajes de programación de manera local.**
- **Lenguajes:** Python (FastAPI), Bash/Scripting básico.
- **Herramientas de Apoyo:** Draw.io / Mermaid.js (Diagramación), Swagger UI (Pruebas de API), Nmap (Auditoría), Docker (Contenerización), GitHub Actions (Orquestación).

Estrategia de Calificación

El Proyecto Final Integrador se evalúa de manera progresiva. Cada laboratorio tiene un valor máximo de **5.0**. Al final del seminario, el promedio de los 4 laboratorios representará la nota total del proyecto.

Rúbrica Laboratorio 1: Despliegue Base y Arquitectura (Fase 1)

Criterio de Evaluación	Descripción del Desempeño Esperado	Puntuación Máxima
1. Despliegue en la Nube (Codespaces)	El estudiante hace el <i>fork</i> correctamente, inicializa el entorno y la API (FastAPI) se ejecuta sin errores. Se proporciona la URL pública funcional y se visualiza el Swagger UI (/docs).	2.0
2. Diagrama Arquitectónico (As-Is)	El diagrama representa claramente las capas de la aplicación (cliente, red, servidor, API). Usa notación estándar (Draw.io / Mermaid) y es fácil de interpretar.	2.0
3. Estructura y Documentación	El repositorio se mantiene ordenado. El archivo README.md incluye los nombres de los integrantes y el enlace al diagrama.	1.0
Total Laboratorio 1		5.0

Rúbrica Laboratorio 2: Auditoría y Ciberseguridad (Fase 2)

Criterio de Evaluación	Descripción del Desempeño Esperado	Puntuación Máxima
1. Auditoría y Análisis de Vulnerabilidades	Documenta correctamente el escaneo de puertos (ej. usando Nmap). Identifica y explica por qué el endpoint <code>/datos-sensibles</code> representa una vulnerabilidad crítica de confidencialidad.	1.5
2. Implementación de Hardening (Código)	Modifica <code>main.py</code> de forma exitosa implementando un control de acceso (ej. Token o Auth básica) para proteger el endpoint vulnerable sin romper el resto de la API.	2.0
3. Validación de Seguridad	Demuestra mediante capturas o ejecución en vivo que el endpoint protegido devuelve un código <code>401 Unauthorized</code> al intentar acceder sin credenciales.	1.5
Total Laboratorio 2		5.0

Rúbrica Laboratorio 3: Contenerización y Automatización (Fase 3)

Criterio de Evaluación	Descripción del Desempeño Esperado	Puntuación Máxima
1. Empaquetado Docker (Dockerfile)	Crea un Dockerfile optimizado. La imagen se construye correctamente y la API se ejecuta a través del contenedor (docker run) exponiendo el puerto adecuadamente.	2.0
2. Script de Automatización (Monitoreo)	Desarrolla un script funcional en Python o Bash que consulta el endpoint /status . El código maneja errores y escribe el resultado correctamente en un archivo de texto/log local.	2.0
3. Integración en el Repositorio	El Dockerfile y el script están subidos al repositorio principal con instrucciones claras en el README.md sobre cómo ejecutarlos.	1.0
Total Laboratorio 3		5.0

Rúbrica Laboratorio 4: CI/CD y Sustentación Final (Fase 4)

Criterio de Evaluación	Descripción del Desempeño Esperado	Puntuación Máxima
1. Pipeline CI/CD (GitHub Actions)	Configura un archivo <code>.yml</code> funcional. El flujo de trabajo se ejecuta automáticamente al hacer <i>push</i> y valida la construcción de la imagen o el código con éxito (Check verde).	1.5
2. Diagrama Arquitectónico Final (To-Be)	Actualiza el diagrama de la Fase 1 para incluir las nuevas capas implementadas: el contenedor Docker, el control de seguridad y el flujo de GitHub Actions.	1.0
3. Dominio Técnico y Sustentación	El grupo explica con claridad las decisiones arquitectónicas, demuestra el funcionamiento integral del sistema en vivo y responde asertivamente a preguntas técnicas sobre la evolución del proyecto.	2.5
Total Laboratorio 4		5.0